

Rust 懒人编程：LazyCell 与 LazyLock 的惰性哲学

原创 寻月隐君 寻月隐君 2025年9月17日 08:31 江苏

Rust 懒人编程：LazyCell 与 LazyLock 的惰性哲学

在软件开发中，延迟加载（Lazy Loading）是一种重要的性能优化策略，它避免了不必要的开销，只在数据真正被需要时才进行初始化。Rust 标准库中的 `LazyCell` 和 `LazyLock` 就是这种哲学的完美体现。它们如同“懒人”容器，承诺只在第一次被访问时才进行初始化，从而极大地提升了程序的效率和响应速度。本文将带你深入这两个类型，探索它们在单线程和多线程环境下的不同应用场景。



Rust 多线程：

LazyCell & LazyLock

LazyCell<T, F>

- 首次访问是初始化的值（即首次访问时进行初始化的）。
- 使用 `OnceCell` 时，经常：对于同一个 `OnceCell`，每次调用 `get_or_init` 时都使用相同的函数。`LazyCell` 相当于把这两步合成一步了。也就是在 `new` 的时候你就得传入一个 `Function` 或者一个闭包，这就相当于它把 `T` 和 `F` 绑定在一起，在获得 `&T` 前总是调用 `F`（当然只是在首次访问的时候才会调用这个 `F`）。怎么获得它里面的值，这个整个过程是隐式发生的，对 `LazyCell` 解引用，就能获得它里面的内容。
- 它不是线程安全的。

LazyLock<T, F>

- `LazyCell` 的线程安全版本，可以在 `static` 中使用
- 和 `LazyCell` 一样，在创建的时候，`new` 的时候，提供一个无参数的初始化函数来完成初始化
- 由于这个 `LazyLock` 可以在多线程里面使用，即初始化可能会被多个线程同时调用，如果在另一个初始化过程正在运行的时候调用了引用操作，那么调用线程会被阻塞，直到初始化完成为止。

实操

示例一

```
use std::cell::LazyCell;

fn main() {
    let lazy = LazyCell::new(init);
    println!("_____");
    println!("{}", *lazy);
    println!("{}", *lazy);
}

fn init() -> i32 {
    println!("initializing...");
    23
}
```

这段代码展示了 `LazyCell` 的“惰性初始化”特性。`LazyCell` 就像一个“懒人”容器，它不会在创建时就立即执行初始化逻辑，而是等到你第一次真正需要使用它的值时，才会去调用 `init` 函数来生成这个值。

代码中，我们首先用 `LazyCell::new(init)` 创建了一个 `lazy` 实例，但此时 `init` 函数并没有被执行，所以控制台也没有打印任何东西。直到 `println!("{}", *lazy)` 这一行，我们通过解引用 `*lazy` 试图访问它的值，`LazyCell` 才会执行闭包 `init` 函数来完成初始化。因此，`"initializing..."` 只会被打印一次。在随后的 `println!("{}", *lazy)` 中，`LazyCell` 因为有了值，就会直接返回，不再执行 `init` 函数。

运行

```

→ cargo run
   Compiling cell v0.1.0 (/Users/qiaopengjun/Code/Rust/RustJourney/cell)
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.47s
   Running `target/debug/cell`

-----
initializing...
23
23

```

这段测试结果清晰地展示了 `LazyCell` 的 **惰性初始化** 特性。

当你运行代码时，程序首先打印了 `-----`。然后，在执行 `println!("{}", *lazy)` 时，`LazyCell` 发现自己还没有值，所以它才第一次调用了 `init()` 函数来生成数据。这就是为什么你看到了 `initializing...` 被打印出来，紧接着是 `init()` 函数返回的值 `23`。之后，当你第二次调用 `println!("{}", *lazy)` 时，`LazyCell` 已经有值了，所以它直接返回了之前存储的 `23`，而 **没有** 再次调用 `init()`。这证明了 `LazyCell` 的初始化过程只发生一次，从而实现了高效的惰性加载。

示例二

```

use std::{sync::LazyLock, thread};

static NUMBER: LazyLock<i32> = LazyLock::new(|| {
    println!("initializing...");
    100
});

fn main() {
    let handles: Vec<_> = (0..5)
        .map(|_| {
            thread::spawn(|| {
                println!("Thread sees NUMBER = {}", *NUMBER);
            })
        })
        .collect();

    for handle in handles {
        handle.join().unwrap();
    }
}

```

这段代码展示了 `LazyLock` 的一个主要特性：**线程安全的惰性初始化**。你可以把它想象成一个全局的、被“锁住”的懒人盒子，里面的东西只会在第一次被需要时，由唯一一个线程来安全地初始化。

代码中定义了一个静态变量 `NUMBER`，它是一个 `LazyLock` 类型。当多个线程同时启动并

尝试访问 `*NUMBER` 时, `LazyLock` 的神奇之处就体现出来了:

- 第一次访问: 尽管有多个线程同时在运行, 但只有一个线程会成功地进入“初始化”阶段, 执行 `println!("initializing...")` 并将值 `100` 存入 `NUMBER`。
- 后续访问: 其他所有线程都会等待这个初始化过程完成。一旦 `NUMBER` 被赋值, 它们会立即获取到 `100`, 而不会再执行任何初始化逻辑。

最终, 你会发现 `initializing...` 只会被打印一次, 而所有线程都能正确地看到 `NUMBER` 的值是 `100`。这确保了在多线程环境中, 昂贵的初始化操作只发生一次, 同时避免了数据竞争。

运行

```
→ cargo run
   Compiling cell v0.1.0 (/Users/qiaopengjun/Code/Rust/RustJourney/cell)
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.59s
   Running `target/debug/cell`
initializing...
Thread sees NUMBER = 100
Thread sees NUMBER = 100
Thread sees NUMBER = 100
Thread sees NUMBER = 100
Thread sees NUMBER = 100
```

这段运行结果完美地展示了 `LazyLock` 在多线程环境下的 **线程安全惰性初始化** 能力。

虽然代码创建了五个线程去并发地访问 `NUMBER`, 但 `LazyLock` 确保了初始化闭包 (`initializing...`) 只被执行了一次。这是因为 `LazyLock` 在底层使用了同步原语, 它会锁定初始化过程, 只允许第一个到达的线程执行初始化逻辑, 而其他线程则会阻塞等待。一旦初始化完成, 所有线程都能安全地读取到同一个值 `100`。因此, 你看到 `initializing...` 只打印了一次, 而 `Thread sees NUMBER = 100` 却打印了五次, 这证明了 `LazyLock` 成功地实现了昂贵操作的一次性初始化, 并让所有线程共享了结果, 同时避免了数据竞争。

示例三

```
use std::{sync::OnceLock, thread};

// LazyLock - OnceLock new + get_or_init
// LazyCell - OnceCell

static NUMBER: OnceLock<i32> = OnceLock::new();

fn main() {
    let handles: Vec<_> = (0..5)
        .map(|i| {
            thread::spawn(move || {
                if i % 2 == 0 {
```

```

        let _ = NUMBER.set(2);
    } else {
        let _ = NUMBER.set(1);
    }
    println!("Thread sees NUMBER = {}", NUMBER.get().unwrap());
})
})
.collect();

for handle in handles {
    handle.join().unwrap();
}
}

```

这段代码展示了 **OnceLock** 在多线程环境下的“只设置一次”特性。尽管五个并发线程都在试图给静态变量 **NUMBER** 赋值（有些想设为 **2**，有些想设为 **1**），但 **OnceLock** 的原子性保证了 **只有一个** 线程会成功完成赋值操作。其他所有线程的 **set** 调用都会失败，并且不会改变 **OnceLock** 中已经存在的值。最终，所有线程都会通过 **get().unwrap()** 获取到并打印出**同一个值**，这证明了 **OnceLock** 成功地保护了共享数据的单次初始化，防止了竞争条件。

运行

```

→ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
    Running `target/debug/cell`
Thread sees NUMBER = 2
Thread sees NUMBER = 2
Thread sees NUMBER = 2
Thread sees NUMBER = 2
Thread sees NUMBER = 2

```

这段运行结果完美地展示了 **OnceLock** 的“只设置一次”核心特性，以及多线程环境下的 **非确定性**。

虽然你创建了五个线程，并且它们的逻辑是根据索引 **i** 试图给 **NUMBER** 赋值为 **2** 或 **1**，但 **OnceLock** 在底层确保了 **只有一个** 线程的 **set** 操作会成功。由于线程调度是不可预测的，哪个线程（**i** 为偶数还是奇数）会先运行并成功设置值，这是不确定的。因此，**NUMBER** 的最终值可能是 **2**，也可能是 **1**。一旦某个线程成功设置了值，其他线程的 **set** 调用都会悄悄失败，而所有线程在读取 **NUMBER** 时都会得到那个唯一的、已经被成功设置的值。这个结果（五个线程都打印 **2**）表明在这次运行中，某个偶数 **i** 对应的线程首先成功地将 **NUMBER** 设置为了 **2**。

结果大概率是2，也有可能是1，因为i是从0开始的。

总结

`LazyCell` 和 `LazyLock` 是 Rust 优雅处理初始化问题的两大基石。它们的设计哲学是：只做必要之事，并且只做一次。

- `LazyCell` 是单线程环境下的理想选择，尤其适用于需要进行昂贵或复杂计算，但又不确定是否会被使用的场景。它让初始化过程变得透明，你只需像操作普通值一样去解引用它，剩下的就交给 `LazyCell` 去处理。
- `LazyLock` 将这一优势扩展到了多线程世界。它通过内部的同步机制，确保了全局静态变量在被多个线程首次访问时，初始化闭包只会执行一次。这不仅解决了多线程下的数据竞争问题，还保证了性能，因为它避免了重复的初始化开销，并保证了所有线程都能安全地共享一个初始化后的值。

简而言之，当你的程序需要按需创建资源时，`LazyCell` 和 `LazyLock` 提供了简洁、安全且高效的解决方案。掌握它们，你就掌握了 Rust 中惰性初始化的精髓。

参考

- <https://doc.rust-lang.org/reference/introduction.html>
- <https://rustcc.gitbooks.io/rustprimer/content/>
- <https://rustwiki.org/zh-CN/cargo/commands/cargo-run.html>

点个“赞”让我知道你喜欢，点个“推荐”让更多「寻月者」看到。



寻月隐君

🌟 寻月隐君 —— 技术的光，未来的道 关注《寻月隐君》，专注Web3、Python...

345篇原创内容

公众号



寻月隐君

喜欢作者

[上一篇](#)

[Rust 并发编程利器：
OnceCell 与 OnceLock 深度解析](#)

[下一篇](#)

[Rust 入门精髓：详解 Enum 的三种魔法，从
模式匹配到状态管理](#)

作者提示: 个人观点，仅供参考